

Parallelizing Seq2Seq on GPU and Comparing with CPU-Based Seq2Seq

Muhammad Saad, M2 SIAM

Student ID: 12419197

Abstract

The investigation of this report focuses on Sequence-to-Sequence (Seq2Seq) model parallelization which is a vital deep learning architecture particularly important for both neural machine translation operations and sequence generation tasks. The research aims to analyze the performance capability of GPU parallelization algorithms with Seq2Seq models to enhance training speed. Our goal is to gain significant speed improvements during training by parallelizing fundamental functions that include recurrent neural network hidden state updates matrix multiplications and softmax calculations because GPUs possess built-in parallel processing features which we will utilize.

A Seq2Seq model comprises two essential elements consisting of an encoder component while its decoder subdivision utilizes the context vector to produce the output sequence. Our execution methodology targets critical computational sections from the components because this strategy enables peak GPU resource use while shortening training duration.

The GPU system running the Seq2Seq model presented a substantial shortening of training times per epoch against its CPU execution. The achievement of maximum performance was limited through poor GPU occupancy because of incorrect CUDA kernel configuration options for grid and block sizes. The GPU-based processing speed-up failed to achieve good model convergence because of these performance bottlenecks. The CPU-based system maintained stable loss reduction patterns through all the epochs although its computation speed remained slower compared to GPU implementation. The benefits GPUs present to massive machine learning operations depend heavily on proper tuning of grid dimensions alongside block partitioning strategies to maximize GPU computational potential thus speeding up model convergence.

Design Methodology

This project adopts parallelization methods to optimize Seq2Seq model operation on Graphics Processing Units (GPUs) which enables the utilization of their deep learning capabilities. Machine translation experts recognize the Seq2Seq model through two main components which form an encoder-linking to decoder system. The training process includes two parts that benefit from optimization techniques to perform efficiently using GPUs. An encoder identifies input sequence data to produce context vector output while the decoder produces output sequence data using this context.

This design features CUDA (Compute Unified Device Architecture) together with Numba to perform parallel computation on the GPU devices while focusing on these operations:

The encoder alongside the decoder depends on matrix multiplications for determining hidden states during operation. The processing requirements of these operations are high therefore delivering exceptional prospects for parallel computing.

This design utilizes **CUDA** (Compute Unified Device Architecture) and **Numba** to enable parallel computation on the GPU, targeting the following key operations:

1. **Matrix Multiplications:** The encoder and decoder execute matrix multiplications as part of determining hidden states. This type of calculation proves to be power-intensive while simultaneously creating ideal conditions for parallel processing.

2. **RNN Hidden State Updates:** The Recurrent Neural Networks (RNNs) in the encoder and decoder require hidden state updates, which are traditionally sequential for a single sequence. However, when processing multiple sequences in a batch, these updates can be performed concurrently.
3. **Softmax Function:** The softmax operation is essential for producing probability distributions from the model's logits in the decoder. Since each sequence in the batch can be processed independently, softmax computations lend themselves to parallel execution.
4. **Loss Calculation:** The softmax output and the actual target values are used to compute cross-entropy loss. Additionally, this can be parallelized across batch dimensions.

Grid and Block Configuration for CUDA Execution

These operations are parallelized using CUDA in the GPU implementation. CUDA's execution mechanism employs a master-slave methodology. While the slave threads (on the GPU) carry out the parallelized processes, the master thread (on the CPU) manages the entire workflow by preparing the data and starting the computational tasks. Task scheduling and data flow are managed by the master (CPU).

- The **master (CPU)** controls the flow of data and task scheduling.
- The **slaves (GPU threads)** perform the core computations in parallel.

CUDA provides flexibility through which developers can optimize their code based on hardware capabilities including block size and grid size parameters that proved significant for GPU performance assessment in this research.

Overview of the Seq2Seq Model

The Seq2Seq model demonstrates exceptional capability for machine translation operations since both the input and output consist of token sequences (such as sentences across languages). The Seq2Seq model contains two main parts that function together:

- **Encoder:**
 - Each token of the input sequence gets processed by the encoder. The encoder maintains internal processing for every token appearing in the input data. The encoder completes its processing of the entire sequence to generate a context vector with compressed information that will be transmitted to the decoder.
 - Mathematically, the update of the encoder's hidden state at each timestep t can be expressed as:

$$h_t = f_{\text{encoder}}(x_t, h_{t-1})$$

where f_{encoder} is the activation function, and the initial hidden state h_0 is typically initialized to zeros.

- **Decoder:**
 - Through the context vector received from the encoder, the decoder generates the output sequence token by token. The auto-regressive mechanism of the decoder applies to output sequences by using context vectors plus previously generated tokens for each

successive token prediction.

- Mathematically, the decoder's state and output at each timestep t are updated as:

$$s_t = f_{\text{decoder}}(y_{t-1}, s_{t-1}, c)$$

$$p(y_t | y_{<t}, x) = g(s_t)$$

where f_{decoder} is the decoder's activation function, and $g(s_t)$ maps the hidden state to a probability distribution over the vocabulary.

Parallelism Identification

The optimization of the Seq2Seq model depends heavily on the parallelization of its operations. There are essential operations that show improved performance when executed in parallel:

1. Matrix Multiplications:

- The update of hidden states makes extensive use of matrix multiplications throughout both the encoder and decoder components. Batch operations in these calculations proceed independently along the sequence dimension because each sequence computation remains separate from others.

2. RNN Computations:

- Although RNN computations are sequential for a single sequence (since the hidden state at timestep t depends on $t-1$), the RNN computations for different sequences in the batch can be computed in parallel.

3. Softmax and Loss Calculation:

- The normalization performed by softmax and the cross-entropy loss calculations operate across multiple sequences per batch for effective training of large datasets.

Master and Slave Execution

Using CUDA master-thread coordination takes place through CPU-GPU task partitioning with the master thread managing workflow direction and GPU computation scheduling and the GPU slave threads performing parallel computations.

Master Execution (CPU):

- The CPU as the master thread does the following:
 - Data preparation and batching:** The dataset is loaded first, pre-processed second, and then split up into smaller batches.
 - Model initialization:** This sets up the model's weights and biases, among other factors.
 - Data transfer:** For processing, it transfers the mini batches from the CPU to the GPU.
 - Kernel launch configuration:** This kernel configures the block and grid structures for GPU parallel execution.
 - Kernel invocation:** It starts the kernels for operations like loss computation, RNN updates, and matrix multiplication.
 - Result retrieval:** Following kernel execution, it returns the GPU's output to the CPU.

Slave Execution (GPU):

- The GPU threads serve as slave threads that execute computations through parallel processing. Each independent thread operates concurrently throughout the workload while performing its designated portion. The main parallel computing operations within this system consist of these stages:
 - **Matrix Multiplications:** All of the sequences in the batch can be processed in parallel since each thread does a portion of the matrix multiplication.
 - **RNN Hidden State Updates:** Before performing matrix multiplication operations and applying non-linear transformation functions, the threads calculate hidden states for each sequence in the batch.
 - **Softmax:** Every thread computes the softmax normalize for the logits of a single sequence in order to ensure that the final probabilities are computed correctly in parallel for each sequence in the batch.
 - **Loss Calculation:** The cross entropy computation is performed independently on each sequence, and loss calculations are performed in parallel.

GPU Communication:

- High performance depends on effective data transfer between the CPU and GPU as well as among GPU components. The GPU stores its three main memory types in global memory that contains model data and parameters and output results alongside shared memory inside thread blocks which holds faster-accessible intermediate data such as reduction partial sums.

Parallel Functions Implemented

Several critical functions in the Seq2Seq model were parallelized for efficient GPU computation:

1. **fast_tanh(x):**
 - The tanh activation function receives optimization to operate in parallel for all elements within the input which accelerates the implementation of non-linear transformations.
2. **softmax_kernel(x, output):**
 - To calculate the probability distribution over several sequences in the batch simultaneously, the softmax function is parallelized. In order to do this, the logits must be exponentiated, normalized, and the maximum value for numerical stability determined.
3. **fused_rnn_kernel(x, W_ih, h_prev, W_hh, b_ih, b_hh, h_next):**
 - For both input-to-hidden and hidden-to-hidden operations, this kernel manages the fused RNN hidden state update in parallel. For efficiency, matrix multiplications and bias adds are combined into a single operation.
4. **embedding_kernel(embeddings, indices, output):**
 - Maps each input token index to its matching embedding in parallel, hence parallelizing the embedding lookup process.
5. **linear_layer_kernel(inputs, weights, bias, output):**
 - Carries out the linear transformation that the output layer of the decoder uses. It adds the bias, calculates the matrix multiplication between the output weights and the hidden state, and is parallelized to manage several sequences in the batch.
6. **cross_entropy_kernel(probs, targets, loss):**
 - The cross-entropy loss computation stretches across all sequences within each batch to allow an efficient computation of gradients during backpropagation.

Results

The Seq2Seq model operated using a dataset containing 68,249 French words together with 68,230 English words. The NLTK tool tokenized the dataset before enabling its indexing and the training process occurred during 10 epochs. Training time together with loss development served as the evaluation metrics that showcased the performance between GPU and CPU implementations.

GPU Training Results

- **Training Time:** The GPU implementation resulted in reduced training time, taking about **30 minutes** per epoch and completing 10 epochs in **5 hours**.
- **Loss Progression:** Despite the speedup, the GPU implementation showed increasing loss, which reached **14,273.03** (learns nothing) by the 10th epoch. This indicated **training divergence**.

CPU Training Results

- **Training Time:** The CPU implementation, though much slower, took approximately **70 minutes** per epoch, totaling **9.34 hours** for 8 epochs.
- **Loss Progression:** The CPU's loss stayed fixed (learns nothing) at **11.0721**, which means that behavior stayed the same but model learning progress was slow.

Comparison of GPU and CPU Performance

- **GPU:** The GPU implementation was faster but did not converge effectively due to **low occupancy** and inefficient grid/block configuration.
- **CPU:** The CPU implementation was slower but showed more consistent performance with stable loss progression.

Table 1: Training Times and Losses for GPU and CPU

Epoch	GPU Avg Loss	GPU Time (s)	CPU Avg Loss	CPU Time (s)
1	746.02	1794.61	11.0720	4402.37
2	2248.95	1822.15	11.0721	4286.30
3	3751.96	1818.31	11.0721	4289.26
4	5254.97	1800.59	11.0721	4295.86
5	6757.98	1790.28	11.0721	4297.76
6	8260.99	1773.26	11.0722	4313.53
7	9764.00	1772.47	11.0722	4304.01
8	11267.01	1774.25	11.0722	4308.68
9	12770.02	1770.54	NA	NA
10	14273.03	1763.09	NA	NA

Conclusion

The main goal of this report involved parallelizing the Seq2Seq model followed by GPU and CPU performance testing. The GPU implementation achieved substantial speedup although low occupancy negatively affected its overall performance. The CPU implementation delivered predictable results at slower speeds although it performed slower than both other modes.

The report demonstrates why GPU configuration optimization and kernel design selection remain essential to achieve maximum parallel system performance. Improving GPU occupancy and developing data parallelism and model parallelism techniques should be the main areas for future development to enhance training performance. The complete objective is to optimize model convergence together with computational speed performance for maximum efficiency on large-scale Seq2Seq operations.